МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Инженерный бизнес и менеджмент”
Кафедра “Промышленная логистика (Бизнес-информатика)”



Отчет по лабораторной работе ЛР2 по курсу

«Парадигмы и конструкции языков программирования»

Тема: «Объектно-ориентированные возможности языка Python»

18 вариант

Выполнил:
Студент группы ИБМ3-34Б

Норжилова Н. З.
Преподаватель:
Бурмистрова М.Ю

Правдина А.Д.

Задание:

1. Необходимо создать виртуальное окружение и установить в него хотя бы один внешний пакет с использованием `pip install <пакет>`. Установка пакета в системный Python (без активации venv) считается невыполнением задания.

2. Необходимо разработать программу, реализующую работу с классами. Программа должна быть разработана в виде консольного приложения на языке **Python 3**.

3. Все файлы проекта (кроме основного файла `main.py`) должны располагаться в пакете `lab_python_oop`.
4. Каждый из нижеперечисленных классов должен располагаться в **отдельном файле** пакета `lab_python_oop`.

18 вариант. Написать программу, в которой описана иерархия классов: печатное издание (журнал, книга, учебник). Базовый класс должен иметь метод вывода на экран характеристик экземпляра, получение категории издания, вычисление количества печатных знаков в зависимости от количества иллюстраций (принять количество иллюстраций 10%, 15%, 25% от общего количества страниц соответственно), проверка на периодичность.

Текст программы на Python:

Main.py

```
import emoji
from lab_python_oop.magazine import Magazine
from lab_python_oop.book import Book
from lab_python_oop.textbook import Textbook

def main():
    N = 18

    while True:
        print(emoji.emojize("\n:books: ВЫБЕРИТЕ ОБЪЕКТ ДЛЯ ДЕМОНСТРАЦИИ
:books:"))
        print("1. Журнал (Magazine)")
        print("2. Книга (Book)")
        print("3. Учебник (Textbook)")
        print("0. Выход")

        choice = input("Ваш выбор: ")

        if choice == '0':
            print(emoji.emojize(":waving_hand: До встречи!"))
            break
        elif choice == '1':
            pub = Magazine("Наука и жизнь", N * 10.5, "Знание", N)
        elif choice == '2':
```

```
            pub = Book("Война и мир", N * 50.0, "Эксмо", "Л.Н. Толстой")
        elif choice == '3':
            pub = Textbook("Алгебра", N * 45.0, "Просвещение", "Макарычев",
"Математика")
        else:
            print(emoji.emojize(":cross_mark: Неверный ввод! Попробуйте снова."))
            continue

        print("\n" + "="*40)
        print(emoji.emojize(f":white_check_mark: {pub}"))

        print(f"Категория: {pub.get_category()}")
        print(f"Периодическое издание: {'Да' if pub.is_periodic() else 'Нет'}")
        print("="*40)

        try:
            print("\nПроверка валидации (пробуем установить отрицательную
цену):")
            pub.price = -100
        except ValueError as e:
            print(f"Ошибка перехвачена: {e}")

if __name__ == "__main__":
    main()
```

printed_publication.py

```
from abc import ABC, abstractmethod

class PrintedPublication(ABC):
    def __init__(self, title: str, price: float, publisher: str):
        self.__title = None
        self.__price = None
        self.__publisher = None




        self.title = title
        self.price = price
        self.publisher = publisher

    @property
    def title(self) -> str:
        """Название издания"""
        return self.__title

    @title.setter
    def title(self, value: str):
        if not value or not value.strip():
            raise ValueError("Название не может быть пустым!")
        self.__title = value
```

```
    @property
    def price(self) -> float:
        """Цена издания"""
        return self.__price

    @price.setter
    def price(self, value: float):
        if value < 0:
            raise ValueError("Цена не может быть отрицательной!")
        self.__price = value


    @property
    def publisher(self) -> str:
        """Издательство"""
        return self.__publisher

    @publisher.setter
    def publisher(self, value: str):
        if not value or not value.strip():
            raise ValueError("Издательство не может быть пустым!")
        self.__publisher = value


    def is_periodic(self) -> bool:
        """Проверка на периодичность. По умолчанию False (не переодическое)."""
        return False

    @abstractmethod
    def get_category(self) -> str:
        """Получение категории издания (абстрактный метод)"""
        pass

    def __str__(self) -> str:
        """Строковое представление объекта"""
        return (f"[{self.get_category()}] Название: {self.title}, "
                f"Цена: {self.price} руб., Издательство: {self.publisher}")
```

book.py

```
from lab_python_oop.printed_publication import PrintedPublication

class Book(PrintedPublication):
    def __init__(self, title: str, price: float, publisher: str, author: str):
        super().__init__(title, price, publisher)
        self.__author = None
        self.author = author

    @property
```

```
    def author(self) -> str:
        """Автор книги"""
        return self.__author

    @author.setter
    def author(self, value: str):
        if not value or not value.strip():
            raise ValueError("Имя автора не может быть пустым!")
        self.__author = value

    def get_category(self) -> str:
        return "Книга"

    def __str__(self) -> str:
        base_str = super().__str__()
        return f"{base_str}, Автор: {self.author}"
```

magazine.py

```
from lab_python_oop.printed_publication import PrintedPublication

class Magazine(PrintedPublication):
    def __init__(self, title: str, price: float, publisher: str, issue_number:
int):
        super().__init__(title, price, publisher)
        self.__issue_number = None
        self.issue_number = issue_number

    @property
    def issue_number(self) -> int:
        """Номер выпуска журнала"""
        return self.__issue_number

    @issue_number.setter
    def issue_number(self, value: int):
        if not isinstance(value, int) or value <= 0:
            raise ValueError("Номер выпуска должен быть положительным целым
числом!")
        self.__issue_number = value

    def get_category(self) -> str:
        return "Журнал"

    def is_periodic(self) -> bool:
        """ПЕРЕОПРЕДЕЛЕНИЕ: Журналы периодические"""
        return True

    def __str__(self) -> str:
        base_str = super().__str__()
        return f"{base_str}, Номер выпуска: {self.issue_number}"
```

textbook.py

```python
from lab_python_oop.book import Book

class Textbook(Book):
    def __init__(self, title: str, price: float, publisher: str, author: str,
subject: str):
        super().__init__(title, price, publisher, author)
        self.__subject = None
        self.subject = subject

    @property
    def subject(self) -> str:
        """Учебный предмет"""
        return self.__subject

    @subject.setter
    def subject(self, value: str):
        if not value or not value.strip():
            raise ValueError("Предмет не может быть пустым!")
        self.__subject = value

    def get_category(self) -> str:
        return "Учебник"

    def __str__(self) -> str:
        base_str = super().__str__()
        return f"{base_str}, Предмет: {self.subject}"
```

Вывод программы:

📚 ВЫБЕРИТЕ ОБЪЕКТ ДЛЯ ДЕМОНСТРАЦИИ📚📚

1. Журнал (Magazine)

2. Книга (Book)

3. Учебник (Textbook)

0. Выход

Ваш выбор: 1

==========

Название: Наука и жизнь, Цена: 189.0 руб., Издательство: Знание, Номер выпуска: 18

Категория: Журнал

Периодическое издание: Да

==========

📰 ВЫБЕРИТЕ ОБЪЕКТ ДЛЯ ДЕМОНСТРАЦИИ📚📚

1. Журнал (Magazine)

2. Книга (Book)

3. Учебник (Textbook)

0. Выход

Ваш выбор: 2

==========

Название: Война и мир, Цена: 900.0 руб., Издательство: Эксмо, Автор: Л.Н. Толстой

Категория: Книга

Периодическое издание: Нет

==========

📄 ВЫБЕРИТЕ ОБЪЕКТ ДЛЯ ДЕМОНСТРАЦИИ📚📚

1. Журнал (Magazine)

2. Книга (Book)

3. Учебник (Textbook)

0. Выход

Ваш выбор: 3

==========

Название: Алгебра, Цена: 810.0 руб., Издательство: Просвещение, Автор: Макарычев, Предмет: Математика

Категория: Учебник

Периодическое издание: Нет

==========

📚 ВЫБЕРИТЕ ОБЪЕКТ ДЛЯ ДЕМОНСТРАЦИИ📚📚

2. Книга (Book)

3. Учебник (Textbook)

0. Выход

Ваш выбор: 0

👋 До встречи!

Виртуальное окружение и зависимости

Для изоляции зависимостей проекта и предотвращения конфликтов версий с глобальными пакетами системы было создано виртуальное окружение.

Создание и активация окружения:
Виртуальное окружение создавалось в корневой папке проекта с помощью встроенного модуля `venv`. Для этого использовалась команда:
`python -m venv venv`
После создания окружения была выполнена его активация. В операционной системе Windows активация выполняется следующей командой:
`venv\Scripts\activate`

Установка зависимостей:
Был установлен сторонний пакет `emoji` из репозитория PyPI и пакет colorama с помощью пакетного менеджера `pip`:
`pip install emoji`

`pip install colorama`

Фиксация зависимостей (requirements.txt):
Для того чтобы проект можно было легко развернуть на другом компьютере, все установленные пакеты и их версии были сохранены в файл `requirements.txt`. Файл был сгенерирован с помощью команды:
`pip freeze > requirements.txt`

```
colorama==0.4.6
emoji==2.16.0
```